

Design Patterns and Principles

SOLID :- Solid is a set of five object-oriented design Principles introduced by "Robert C. Martin"

- * The purpose of Solid is to create software that is:
- ⇒ Maintainable
 - ⇒ Scalable
 - ⇒ Flexible
 - ⇒ Reusable
 - ⇒ Easy to test.

SOLID Stands for:

- S - Single Responsibility Principle (SRP)
- O - Open closed Principle (OCP)
- L - Liskov Substitution Principle (LSP)
- I - Interface Segregation Principle (ISP)
- D - Dependency Inversion Principle (DIP)

1) Single Responsibility Principle :- A class can perform only a single responsibility.

Why SRP:-

When a class perform multiple tasks :-

- * Difficult to maintain
- * Difficult to test
- * Difficult to debug

Real world Ex:- Hospital.

Doctor → Treat Patient

Receptionist → Handles Appointments

Cashier → Handles Payment

Each person have only a single responsibility

Bad Ex:-

```
class Employee {  
    public void calculateSalary() {  
    }  
    public void saveEmployee() {  
    }  
}
```

Good Examples:-

class Employees

class SalaryServices

Public void CalculateSalary()

class EmpRepository

Public void SaveEmployee()

2) Open closed Principle:-

Software entities should be:-

* open for extensions

* Closed for Modifications!

* why OCP:-

When new functionality comes,

we should add new code instead of changing existing code.

Prblm Ex:-

* Developer Salary

* Tester Salary

Later: we have to add

* Manager Salary

* HR Salary

* Every time we edit existing code May break & more bugs.

Use: inheritance
Polymorphism
Interface.

Adv:- Easy Enhancement
* Less bugs
* Better Scalability.

Ex:- BAD
class SalaryCalculator {

double calculate (String role) {

if (role.equals("Developer"))

return 50000;

if (role.equals("Tester"))

return 40000;

return 0;

}

}

=> when a new roles
come like HR,
Manager
Need to Modify

Good:-

public class Employee {

public void calcSalary() {

}

}

class Developer extends Employee {

double calcSalary() {

return 50000;

}

}

class Tester extends Employee {

double calcSalary() {

return 40000;

}

}

Add functionality by extend classes not modify existing code.



3) Liskov Substitution Principle:-

Object of the child classes can be replaced by the object of parent class without affecting correctness.

Why LSP:-

Inheritance should not break functionality.

Ex:-

⇒ Parent class

* Employee

⇒ Children classes

* Developer

* Tester

* Manager

* If code expects Employee:

Employee Em = new Developer();

⇒ Program should work.

Ex:-

```
class Employee {
```

```
    void work() {
```

```
        System.out.println("working");
```

```
    }
```

```
class Developer extends Employee {
```

```
    void work() {
```

```
        System.out.println("coding");
```

```
    }
```

```
}
```

⇒ Employee Em = new Developer();

Em.work();

It works correctly.

Adv:-

* Proper inheritance

* Better Polymorphism

* Reliable code

4) Interface Segregation Principles:-

Client should not be forced to depend on methods they don't use.

Why ISP:- Large interfaces create unnecessary implementation.

Pblm:- EmployeeOperations

Contains

Code()

Test()

manage()

⇒ Developer only code. So still force to implement

Test()

manage()

Adv:- Flexible design
Easy maintenance
Loose coupling.

BAD Examples:-

```
Interface EmployeeOps {  
    void code();  
    void test();  
    void manage();  
}
```

Developer is forced to implement:

```
public void manage() {  
}
```

Even though developer can't work.

Good:-

```
interface CodeOps {  
    void code();  
}  
interface TestOps {  
    void test();  
}  
interface ManageOps {  
    void manage();  
}
```

Developer:-

class Developer implements coder {

public void code() {

System.out.println("coding");

}

}

he works only what he want he not forced to
work on methods he can't do.

* 5) Dependency Inversion Principle:-

High level module should not depend on low-level
modules.

⇒ Both should depend on abstraction

Why DIP:-

Reduces tight coupling.

Makes application flexible.

Adv:- Loose coupling

Easy testing

Easy replacement

Better Scalability

BAD Example:-

class EmailService {

void send() {

System.out.println("Email sent");

}

}

class Notification {

EmailService email = new EmailService();

void notify() {

email.send();

}

Good Example:-

```
interface MessageService {  
    void send();  
}
```

```
class EmailService implements MessageService {  
    public void send() {  
        System.out.println("Email send");  
    }  
}
```

```
class SMSService implements MessageService {  
    public void send() {  
        System.out.println("SMS send");  
    }  
}
```